

Revisao — Fase 1

Fundacao e Ambiente

Projeto E-Commerce com Microservicos | Dev Junior | Davi x Claude

Sumario

1. O que foi construido nessa fase
2. Bloco 1 — Ambiente WSL e Node.js
3. Bloco 2 — Git e autenticao SSH
4. Bloco 3 — Repositorio e estrutura do projeto
5. Bloco 4 — Docker Compose e bancos de dados
6. Bloco 5 — README e documentacao
7. Comandos essenciais aprendidos
8. Historico de problemas e solucoes
9. Conceitos tecnicos consolidados
10. Checklist de conclusao
11. Proximos passos — Fase 2

1. O que foi construído nessa fase

A Fase 1 não tem código de negócio. O objetivo foi preparar o terreno técnico completo antes de qualquer serviço ser desenvolvido. Ao final dessa fase, você tem um ambiente profissional funcional, um repositório versionado no GitHub e três bancos de dados rodando em containers Docker — prontos para serem usados nas próximas fases.

Entregável	Status
Node.js v22 LTS	Rodando dentro do WSL via nvm
Git + SSH	Configurado e autenticado com o GitHub
Repositório	github.com/DaviMoraismdev/ecommerce-platform
Estrutura monorepo	11 pastas de serviços criadas
.gitignore	Protegendo .env e node_modules
.env.example	Template de variáveis de ambiente
docker-compose.yml	PostgreSQL, MongoDB e Redis orquestrados
README.md	Documentação profissional do projeto
Commits realizados	3 commits no padrão Conventional Commits

2. Bloco 1 — Ambiente WSL e Node.js

O primeiro bloco estabeleceu um conceito fundamental: Windows e WSL sao ambientes isolados. Programas instalados no Windows nao existem dentro do WSL e vice-versa. Todo o projeto roda dentro do WSL porque servidores reais rodam Linux — desenvolver nesse ambiente garante paridade com a producao.

Por que nvm em vez de instalar o Node diretamente?

O nvm (Node Version Manager) permite ter multiplas versoes do Node no mesmo sistema e trocar entre elas com um comando. Projetos diferentes podem exigir versoes diferentes. Instalando o Node direto no sistema, voce fica preso em uma versao e qualquer mudanca exige desinstalar e reinstalar tudo.

Por que Node.js v22 LTS?

LTS significa Long Term Support. Versoes LTS recebem correcoes de seguranca por 3 anos. A versao Current (v24, que estava no Windows) fica sem suporte em 6 meses se nao virar LTS. Para projetos que duram meses, LTS e a escolha correta.

Comandos utilizados neste bloco

Comando	O que faz
<code>curl -o- https://...nvm.../install.sh bash</code>	Baixa e executa o instalador oficial do nvm
<code>source ~/.bashrc</code>	Recarrega o terminal para reconhecer o nvm
<code>nvm --version</code>	Confirma que o nvm foi instalado
<code>nvm install 22</code>	Instala o Node.js v22 LTS dentro do WSL
<code>nvm use 22</code>	Ativa a versao 22 na sessao atual
<code>nvm alias default 22</code>	Define v22 como padrao permanente
<code>node -v && npm -v</code>	Confirma versoes instaladas

3. Bloco 2 — Git e autenticacao SSH

Todo commit gravado no historico do Git carrega o nome e e-mail do autor permanentemente. Configurar esses dados foi o primeiro passo. O segundo foi estabelecer uma conexao segura com o GitHub via SSH para que o WSL consiga fazer push sem digitar senha.

Como o SSH funciona

O SSH gera um par de chaves matematicamente relacionadas. A chave privada fica na sua maquina e nunca sai dela. A chave publica e cadastrada no GitHub. Quando o WSL tenta se conectar, os dois lados trocam uma prova matematica usando esse par. Se bater, a conexao e autorizada sem nenhuma senha trafegar pela rede.

Por que ed25519 e nao RSA?

ed25519 e o algoritmo mais moderno para chaves SSH. Gera chaves menores, e mais rapido e considerado mais seguro que o RSA tradicional. E o padrao recomendado atualmente.

Comandos utilizados neste bloco

Comando	O que faz
git config --global user.name "Nome"	Registra seu nome em todos os commits
git config --global user.email "email"	Registra seu e-mail em todos os commits
git config --global --list	Lista todas as configuracoes globais do Git
ssh-keygen -t ed25519 -C "email"	Gera o par de chaves SSH
cat ~/.ssh/id_ed25519.pub	Exibe a chave publica para copiar ao GitHub
ssh -T git@github.com	Testa a conexao SSH com o GitHub

4. Bloco 3 — Repositorio e estrutura do projeto

O repositorio foi criado vazio no GitHub e clonado no WSL via SSH. A estrutura de pastas do monorepo foi montada antes de qualquer codigo — isso e uma decisao arquitetural importante. Reorganizar pastas depois que servicos estao criados e trabalhoso e confuso.

Por que monorepo?

Monorepo significa um unico repositorio Git contendo todos os servicos. A alternativa seria um repositorio por servico. Para um projeto educacional, o monorepo e melhor porque voce ve o projeto inteiro de uma vez, compartilha configuracoes sem duplicar e o historico de commits conta a historia completa do sistema.

O que e o .gitignore?

O .gitignore e um arquivo que diz ao Git o que nunca deve ser versionado. Os itens mais criticos sao: node_modules (dependencias regeneraveis), .env (credenciais sensiveis) e dist (codigo compilado). Se o .env fosse para o GitHub, bots automaticos varrendo repositorios publicos encontrariam suas senhas em segundos.

O que e o .env.example?

O .env.example e o template publico das variaveis de ambiente. Ele documenta quais variaveis o projeto precisa sem expor os valores reais. Quando alguem clonar o repositorio, roda cp .env.example .env e preenche com as proprias credenciais.

Comandos utilizados neste bloco

Comando	O que faz
mkdir -p pasta/{sub1,sub2}	Cria arvore de pastas em um unico comando
find . -type d sort	Lista todas as pastas do projeto
explorer.exe .	Abre o Gerenciador de Arquivos do Windows na pasta atual
code .	Abre o VS Code conectado ao WSL na pasta atual
git init	Inicializa o Git na pasta (cria .git oculto)
git branch -M main	Renomeia a branch padrao para main
git remote add origin URL	Conecta o repositorio local ao GitHub
git remote -v	Confirma o endereco do repositorio remoto
git add .	Adiciona todos os arquivos ao stage
git status	Mostra o estado atual do repositorio
git commit -m "tipo: mensagem"	Cria um commit com mensagem
git push -u origin main	Envia commits ao GitHub e vincula branches

5. Bloco 4 — Docker Compose e bancos de dados

O Docker Compose permite descrever em um arquivo YAML toda a infraestrutura necessaria. Com um unico comando, tres bancos de dados sobem em containers isolados, com volumes para persistencia e uma rede interna para comunicacao.

Imagem vs Container

A imagem e o molde — define como algo deve ser, mas nao faz nada sozinha. O container e a instancia rodando — a imagem executada de verdade na memoria. A analogia com orientacao a objetos: imagem e a classe, container e o objeto instanciado.

O que e um volume?

Por padrao, tudo dentro de um container e temporario. Se o container for removido, os dados somem. O volume resolve isso: o Docker cria um espaco gerenciado no disco e o mapeia para a pasta interna do container onde os dados sao salvos. Com volume, os dados persistem independente do ciclo de vida do container.

O que e a rede interna (ecommerce-network)?

Containers na mesma rede se enxergam pelo nome, como se fossem hosts na mesma rede local. O auth-service vai se conectar ao PostgreSQL usando o host postgres, nao localhost. Containers em redes diferentes nao se enxergam, o que traz isolamento e seguranc

Mapeamento de portas

O formato HOST:CONTAINER mapeia uma porta do seu WSL para uma porta interna do container. 5432:5432 significa: quando voce se conectar na porta 5432 do WSL, o Docker redireciona para a porta 5432 dentro do container do PostgreSQL.

Banco	Porta	Volume	Usado em
PostgreSQL 15	5432	postgres_data	auth, user, order, payment
MongoDB 7	27017	mongo_data	product-service
Redis 7	6379	redis_data	carrinho, cache, tokens

Comandos utilizados neste bloco

Comando	O que faz
docker --version	Confirma que o Docker esta acessivel no WSL
docker compose up -d	Sobe todos os containers em segundo plano
docker compose ps	Lista os containers e seus status
docker compose down	Para e remove os containers (dados persistem nos volumes)
docker compose logs servico	Exibe os logs de um container especifico

6. Bloco 5 — README e documentacao

O README.md e a porta de entrada do projeto. E a primeira coisa que um recrutador ou desenvolvedor ve ao acessar o repositorio. Um README bem feito comunica profissionalismo antes mesmo de abrir qualquer arquivo de codigo.

O README criado contem: descricao do projeto, diagrama de arquitetura em texto, tabela de tecnologias, pre-requisitos, instrucoes de como rodar localmente, estrutura de pastas e progresso das fases com checkboxes.

-> O README deve ser atualizado a cada fase concluida — marcar o checkbox da fase e adicionar qualquer informacao relevante.

7. Comandos essenciais aprendidos

Referencia rapida de todos os comandos vistos na Fase 1.

Git — fluxo diario

Comando	Uso
git status	Sempre antes de commitar — veja o que vai entrar
git add .	Adiciona tudo ao stage
git add arquivo	Adiciona um arquivo especifico ao stage
git commit -m 'tipo: msg'	Cria o commit
git push	Envia para o GitHub
git log --oneline	Historico resumido de commits
git diff	Mostra o que mudou ainda nao commitado

Docker — operacoes comuns

Comando	Uso
docker compose up -d	Sobe todos os containers
docker compose down	Para e remove os containers
docker compose ps	Lista containers e status
docker compose logs -f servico	Acompanha logs em tempo real
docker compose restart servico	Reinicia um container especifico
docker volume ls	Lista todos os volumes
docker network ls	Lista todas as redes

Ritual de inicio de sessao

1. Abre o Docker Desktop e aguarda ficar ativo
2. Abre o terminal WSL
3. `cd ~/projetos/ecommerce-platform`
4. `docker compose up -d`
5. `docker compose ps` (confirma os 3 containers Up)
6. `code .` (abre o VS Code conectado ao WSL)
7. Pronto para trabalhar

8. Histórico de problemas e soluções

Todos os erros encontrados durante a Fase 1, documentados para referência futura. Quando esses erros aparecerem novamente, você saberá resolver em segundos.

Erro: npm error ENOENT – syscall lstat C:\Users\davim\AppData\Roaming\npm

Causa: Ocorreu durante a instalação do nvm. O instalador encontrou o npm do Windows no PATH do WSL.

Por que aconteceu: O nvm foi instalado corretamente — o erro foi em um passo secundário. Após instalar o Node via nvm, o conflito desapareceu automaticamente.

Solução: Nenhuma. O erro se resolve sozinho ao instalar o Node dentro do WSL.

Erro: error: pathspec 'Estrutura' did not match any file(s) known to git

Causa: Tentativa de rodar git commit -m chore: Estrutura inicial do monorepo sem aspas.

Por que aconteceu: O terminal interpretou cada palavra como um arquivo separado que o Git deveria commitar.

Solução: Adicionar aspas corretamente: git commit -m "chore: mensagem completa aqui"

Erro: WARN[0000] version is obsolete – docker-compose.yml

Causa: O arquivo docker-compose.yml continha version: 3.8 no topo.

Por que aconteceu: Versões recentes do Docker Compose não precisam mais dessa declaração — ela é obsoleta.

Solução: Remover a linha version: 3.8 do topo do arquivo.

9. Conceitos técnicos consolidados

Conceitos que você aprendeu e deve conseguir explicar ao final dessa fase.

WSL2

Windows Subsystem for Linux versão 2. Um Linux completo rodando dentro do Windows, com sistema de arquivos e programas próprios e isolados do Windows.

nvm

Node Version Manager. Gerencia múltiplas versões do Node.js no mesmo sistema, permitindo trocar entre elas sem conflito.

LTS (Long Term Support)

Versões de software com suporte estendido de segurança. Para Node.js, LTS recebe correções por 3 anos. A versão Current recebe por apenas 6 meses.

Chave SSH

Par de chaves criptográficas para autenticação segura. A chave privada fica na sua máquina, a pública é cadastrada no servidor remoto (GitHub). Nenhuma senha trafega.

Monorepo

Repositório único contendo múltiplos projetos ou serviços. Facilita visibilidade global e compartilhamento de configurações.

GitFlow

Modelo de organização de branches: main para produção estável, develop para desenvolvimento, feature/* para novas funcionalidades.

Conventional Commits

Convenção de mensagens de commit com tipo prefixado: feat, fix, chore, docs, test, refactor. Mantém o histórico legível e permite automações.

.gitignore

Arquivo que lista o que o Git deve ignorar. Protege credenciais (.env), dependências (node_modules) e builds (dist) de ir para o repositório.

Imagem Docker

O molde de um container — define o ambiente e os programas, mas não roda sozinha. Analogia: é a classe em OOP.

Container Docker

Instância rodando de uma imagem. Processo isolado com seu próprio sistema de arquivos. Analogia: é o objeto instanciado da classe.

Volume Docker

Espaço de armazenamento gerenciado pelo Docker que persiste dados além do ciclo de vida do container. Sem volume, dados somem ao remover o container.

Rede interna Docker

Rede privada onde containers se enxergam pelo nome. Containers fora da rede não conseguem se comunicar — isolamento por design.

Docker Compose

Ferramenta para orquestrar múltiplos containers com um arquivo YAML. Um comando sobe toda a infraestrutura do projeto.

10. Checklist de conclusao

Todos os criterios abaixo foram atendidos. A Fase 1 esta oficialmente concluida.

[OK] Funcionando

Node v22 no WSL, Docker com 3 containers Up, Git com SSH autenticado

[OK] Testado

Cada passo verificado com output confirmado antes de avancar

[OK] Commitado

3 commits no padrao Conventional Commits enviados ao GitHub

[OK] Documentado

.gitignore, .env.example, docker-compose.yml e README.md criados

[OK] Compreendido

Conceitos validados durante o percurso com perguntas e respostas

[OK] Revisado

Este PDF gerado e salvo em docs/phase-reviews/

11. Proximos passos — Fase 2

A Fase 2 comeca com o servico mais critico do sistema: o auth-service. Ele e o primeiro servico real do projeto e vai definir o padrao de qualidade para todos os outros que virao depois.

O que vamos construir

Um servico Node.js + Express completo com cadastro, login, JWT, refresh token, OAuth2 com Google, controle de papeis e rate limiting.

O que voce vai aprender

Como uma API REST e estruturada, o que e middleware, como o JWT funciona por dentro, o que e hashing de senha com bcrypt, e como o Prisma conecta o Node ao PostgreSQL.

Pre-requisito para comecar

Os 3 containers da Fase 1 devem estar rodando. Rode docker compose ps e confirme status Up antes de comecar qualquer comando da Fase 2.

Primeiro passo da Fase 2

Inicializar o auth-service dentro de services/auth-service com npm init e instalar as dependencias base. Nada antes disso — plano primeiro, codigo depois.